

JUnit

September 21st, 2026

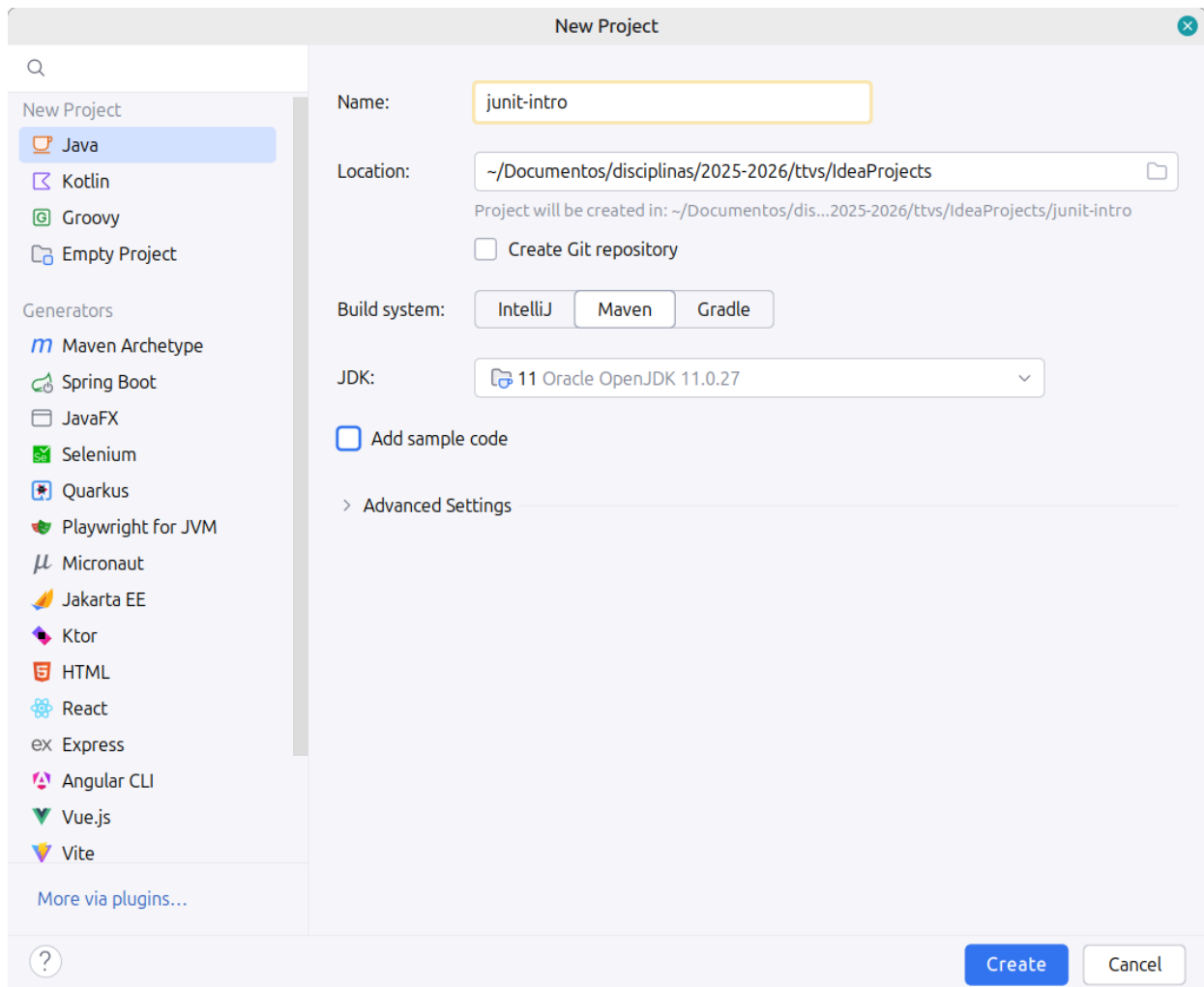
Ana Paiva, Auri Vincenzi, José Campos

[JUnit](#) is a widely-used unit testing framework in the Java ecosystem. With the release of JUnit 5, many new features were introduced, leveraging the capabilities of Java 8 and beyond. This tutorial provides an introduction to unit testing with the JUnit framework, focusing specifically on the features and usage of JUnit 5.

 Tip: In case you have successfully developed unit tests using JUnit in the past, you may skip the first sections of this tutorial and jump to the [Exercises](#) section.

Introduction

Create a new empty Maven project in IntelliJ.



In the project structure `src/main/java` you should first create a package named `pt.up.fe`. Inside this package, create the `Calculator` Java class with the following code, which you want to test.

```
Java
package pt.up.fe;

public class Calculator {

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

A JUnit *test* is a method contained in a class which is only used for testing. This is called a *Test* class. To mark a method as a test method, annotate it with the `@Test` annotation. The annotated method executes the code under test.

In the project structure `src/test/java` you should first create a package named `pt.up.fe`. Inside this package, create the `CalculatorTest` Java class with the following code which defines a minimal test class with one minimal test method for the `Calculator` class.

Java

```
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class CalculatorTest {

    @Test
    public void testMultiply() {
        Calculator calculator = new Calculator();
        assertEquals(20, calculator.multiply(4, 5));
    }
}
```

You can use *assert methods*, provided by JUnit or another assert framework (e.g., [AssertJ](#)), to check an expected result versus the actual result. Such statements are called *oracles*, *asserts*, or *assert statements*.

Before executing the test method, ensure that your `pom.xml` file contains the correct version of the JUnit dependency. We are using version 5.13.4, which is a version compatible with Java 11.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
```

```
<artifactId>junit-intro</artifactId>
<version>1.0-SNAPSHOT</version>

<properties>
  <maven.compiler.source>11</maven.compiler.source>
  <maven.compiler.target>11</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.13.4</version>
    <scope>test</scope>
  </dependency>
</dependencies>
</project>
```

Where should the test be located?

Typically, unit tests are created in a separate source folder to keep the test code separate from the real code. The standard convention from the Apache Maven and Gradle build tools is to use:

- `src/main/java` for Java classes
- `src/test/java` for test classes

JUnit test class naming conventions

Build tools like Apache Maven use a pattern to decide if a class is a test class or not. The following is the list of classes Apache Maven considers automatically during its build:

- `**/Test*.java`, includes all of its subdirectories and all Java filenames that start with Test.
- `**/*Test.java`, includes all of its subdirectories and all Java filenames that end with Test.
- `**/*Tests.java`, includes all of its subdirectories and all Java filenames that end with Tests.
- `**/*TestCase.java`, includes all of its subdirectories and all Java filenames that end with TestCase.

Therefore, it is common practice to use the `Test` or `Tests` suffix at the end of test class names.

JUnit test case naming conventions

Checkout [Jonas Geiregat](#)'s blog post [The subtle Art of Java Test Method Naming](#).

You might also use the `@DisplayName("<Name>")` annotation, where the string `<Name>` will be displayed by the test runner. In contrast to the name of the test case in the source code, `<Name>` can contain spaces to improve readability.

Test life cycle

Each test case undergoes different phases as part of its execution; each phase is represented by an *Annotation*.

@BeforeAll The method marked with this annotation will be executed before any of the **@Test** methods are executed inside the test class.

@BeforeEach The method marked with this annotation will be executed before each **@Test** method in the test class.

@AfterEach The method marked with this annotation will be executed after each **@Test** method in the test class.

@AfterAll The method marked with this annotation will be executed after all the **@Test** methods are executed in the test class.

The *Before* methods (**@BeforeAll**, **@BeforeEach**) perform some initialization actions like setting up test data or environment data, before executing the tests.

The *After* methods (**@AfterAll**, **@AfterEach**) perform clean-up actions like cleaning up the created Environment Data or Test Data.

JUnit instantiates the test class for each method marked with **@Test**. Thus, the method marked with **@BeforeAll** and **@AfterAll** should be marked with static.

The anatomy of a JUnit test

Arrange, Act, and Assert

```
Java
@Test
void test_sum_of_two_numbers() {
```

```

// Arrange
// 🧑 This is where you prepare and initialise all variables and
// objects that are needed by your system under test to work.
double first = 10;
double second = 20;
Calculator calculator = new Calculator();

// Act
// 🧑 In this section you invoke the method under test.
double result = calculator.sum(first, second);

// Assert
// 🧑 This section is used to validate the return value received
// from the sum() method. If the returned value is what is expected,
// then the test case should pass. If the returned value is not what
// was expected, then the test case should fail.
assertEquals(30, result);
}

```

Given, When, and Then

```

Java
@Test
void test_sum_of_two_numbers() {
    // Given
    // 🧑 ... these inputs
    double first = 10;
    double second = 20;
    Calculator calculator = new Calculator();

    // When
    // 🧑 ... the test case executes the following code
    double result = calculator.sum(first, second);

    // Then
    // 🧑 ... the outcome should be
    assertEquals(30, result);
}

```

Assertions and assumptions

JUnit 5 comes with multiple assert statements, which allow you to test your code under test in different ways. You may find the set of assert statements available in JUnit 5 [here](#). All of them are static methods from the `org.junit.jupiter.api.Assertions.*` package.

Assert statements typically allow defining messages which are shown if the test fails. Note: You should provide meaningful messages here to make it easier for the user to identify and fix the problem. This is especially true if someone looks at the problem who did not write the code under test or the test code.

Testing multiple assertions (grouped assertions) with `assertAll`

If an assert fails in a test, JUnit will stop executing the test, and additional asserts are not checked. In case you want to ensure that all asserts are checked, you can `assertAll`.

In this grouped assertion, all assertions are executed, even after a failure. The error messages are also grouped together.

```
Java
@Test
void groupedAssertions() {
    Address address = new Address();
    assertAll("address name",
        () -> assertEquals("John", address.getFirstName()),
        () -> assertEquals("User", address.getLastName())
    );
}
```

If these tests fail, the result looks like the following:

```
None
=> (2 failures)
expected: <John> but was: <null>
expected: <User> but was: <null>
```

Testing for exceptions

Testing that certain exceptions are thrown can be done with the `org.junit.jupiter.api.Assertions.assertThrows()` assert statement. You define the expected `Exception` class and provide code that should throw the exception.

Java

```
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class UserTest {

    private User user;

    @BeforeEach
    void setup() {
        user = new User();
    }

    @Test
    void exceptionTesting() {
        assertThrows(IllegalArgumentException.class, () -> user.setAge("A"));
    }

    @Test
    void exceptionTestingMessage() {
        Throwable exception = assertThrows(IllegalArgumentException.class,
            () -> user.setAge("A"));
        assertEquals("Age must be an Integer.", exception.getMessage());
    }

    // This is date used for the test
    private class User {
        // not really a setter, just here to demonstrate how the test works
        public void setAge(String ageString) {
            try {
```

```

        Integer.parseInt(ageString);
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException("Age must be an Integer.");
    }
}
}
}

```

This lets you define which part of the test should throw the exception. The test will still fail if an exception is thrown outside of this scope.

Defining timeouts in your tests

If you want to ensure that a test fails if it is not finished in a certain amount of time, you can use the `org.junit.jupiter.api.Assertions.assertTimeout()` assert statement. This assertion fails the method if the timeout is exceeded.

```

Java
import static org.junit.jupiter.api.Assertions.assertTimeout;
import static java.time.Duration.ofSeconds;
import static java.time.Duration.ofMinutes;

@Test
void timeoutNotExceeded() {
    assertTimeout(ofMinutes(1), () -> service.doBackup());
}

@Test
void timeoutNotExceededWithResult() {
    Integer doBackupResult = assertTimeout(ofSeconds(1), () -> {
        return service.doBackup();
    });
    assertEquals(200, doBackupResult);
}

```

Which may result in the following when executed

None

=> execution exceeded timeout of 1000 ms by 212 ms

⚠️ Test `timeoutNotExceededWithResult` might be flaky, i.e., it might fail from time to time. It might be that your computer or the test server is busy, and the test execution might take longer than expected.

How to disable tests / Conditional executions

The `@Disabled` or `@Disabled("Why disabled")` annotation marks a test to be disabled. This is useful when the underlying code has been changed and the test case has not yet been adapted, or if the test demonstrates an incorrect behaviour in the code which has not yet been fixed. It is best practice to provide the optional description of why the test is disabled.

```
Java
@Test
@Disabled("Work in progress")
public void check_whether_is_true() {
    throw new RuntimeException("Test should not be executed");
}
```

Alternatively, you can use `Assumptions.assumeFalse` or `Assumptions.assumeTrue` to define a condition for test execution. `Assumptions.assumeFalse` marks the test as invalid if its condition evaluates to `true`. `Assumptions.assumeTrue` evaluates the test as invalid if its condition evaluates to `false`. For example, the following disables a test on Linux:

```
Java
Assumptions.assumeFalse(System.getProperty("os.name").contains("Linux"));
```

This gives `TestAbortedException`, which the test runners evaluate as skipped tests. For example, the following `testMultiplyWithZero` is skipped if executed on Linux.

```
Java
package pt.up.fe;

import static org.junit.jupiter.api.Assertions.assertEquals;
```

```

import org.junit.jupiter.api.Assumptions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.RepeatedTest;
import org.junit.jupiter.api.Test;

public class CalculatorTest {

    private Calculator calculator;

    @BeforeEach
    void setUp() throws Exception {
        calculator = new Calculator();
    }

    @Test
    void testMultiplyWithZero() {

        Assumptions.assumeFalse(System.getProperty("os.name").contains("Linux"));
        assertEquals(calculator.multiply(0,5), 0,
            "Multiple with zero should be zero");
        assertEquals(calculator.multiply(5,0), 0,
            "Multiple with zero should be zero");
    }
}

```

You can also *enable* (**EnabledOnOs**) and *disable* (**DisabledOnOs**) the execution of the **@Test** methods in the test class based on a specific condition. You may find below the same **testMultiplyWithZero** test, but with a different annotation.

```

Java
@Test
@DisabledOnOs(value = OS.LINUX, disabledReason = "Should not run on Linux")
public void testMultiplyWithZero() {
    assertEquals(calculator.multiply(0,5), 0,
        "Multiple with zero should be zero");
    assertEquals(calculator.multiply(5,0), 0,
        "Multiple with zero should be zero");
}

```

```
}  
}
```

Dynamic tests

Repeated Tests

If you have any functionality which has some randomness to it, you may want to create and run that test multiple times to make sure that the functionality is working as expected. JUnit 5 provides the `@RepeatedTest` annotation to fulfil this requirement.

You can repeat the test `N` number of times by passing the number as an argument to the annotation. For example, `@RepeatedTest(5)` will run the test marked with this annotation 5 times.

Java

```
@RepeatedTest(5)  
public void shouldTestContactCreationRepeatedly() {  
    contactManager.addContact("John", "Doe", "0123456789");  
    assertFalse(contactManager.getAllContacts().isEmpty());  
    assertEquals(1, contactManager.getAllContacts().size());  
}
```

which prints out

None

```
repetition 1 of 5(shouldTestContactCreationRepeatedly())  
repetition 2 of 5(shouldTestContactCreationRepeatedly())  
repetition 3 of 5(shouldTestContactCreationRepeatedly())  
repetition 4 of 5(shouldTestContactCreationRepeatedly())  
repetition 5 of 5(shouldTestContactCreationRepeatedly())
```

You can also specify custom names instead of the default repetition 1 of 5, using the name field. For example:

Java

```
@RepeatedTest(value = 5, name = "Repeating test  
shouldTestContactCreationRepeatedly {currentRepetition} out of  
{totalRepetitions}")  
public void shouldTestContactCreationRepeatedly() {  
    contactManager.addContact("John", "Doe", "0123456789");  
    assertFalse(contactManager.getAllContacts().isEmpty());  
    assertEquals(1, contactManager.getAllContacts().size());  
}
```

which prints out

None

```
Repeating test 1 out of 5(shouldTestContactCreationRepeatedly())  
Repeating test 2 out of 5(shouldTestContactCreationRepeatedly())  
Repeating test 3 out of 5(shouldTestContactCreationRepeatedly())  
Repeating test 4 out of 5(shouldTestContactCreationRepeatedly())  
Repeating test 5 out of 5(shouldTestContactCreationRepeatedly())
```

Parameterized tests

JUnit 5 also supports the execution of a single test multiple times, each time with a different input value. Those are called parameterized tests and are marked with the `@ParameterizedTest` annotation.

Below is an example of a non-parameterized test.

Java

```
import static org.junit.jupiter.api.Assertions.*;  
  
public class NotUsingParameterizedTest {  
  
    // Class to be tested  
    class MyClass {  
        // Method to be test  
        public int multiply(int i, int j) {  
            return i * j;  
        }  
    }  
}
```

```

@Test
public void test0() {
    MyClass myClass = new MyClass();
    assertEquals(2, myClass.multiply(1, 2));
}

@Test
public void test1() {
    MyClass myClass = new MyClass();
    assertEquals(15, myClass.multiply(5, 3));
}

@Test
public void test0() {
    MyClass myClass = new MyClass();
    assertEquals(484, myClass.multiply(121, 4));
}
}

```

As we can see, we are testing multiple times with a different set of inputs. We can simplify the test class and, at the same time, test the `multiply` method against a different (and larger) set of inputs.

 Tip: this test class could be further simplified with `@BeforeEach`. The initialization of `MyClass` happens in every test, you could move it to a `@BeforeEach` method. Try it.

You can provide the input to the test in different ways.

MethodSource

We can use `@MethodSource` annotation to provide the input to our Parameterized tests. Using this annotation, we will refer to the method name which returns the values required for our tests as output. The method has to be `static` and must return either a `Collection`, an `Iterator`, a `Stream` or an `Array`. The test method is executed once for every entry in the data source.

 `@BeforeEach` and `@AfterEach` methods are called for parameterized tests as for non-parameterized tests.

```

Java
import static org.junit.jupiter.api.Assertions.*;

```

```

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.MethodSource;

public class UsingParameterizedTest {

    // Class to be tested
    class MyClass {
        // Method to be test
        public int multiply(int i, int j) {
            return i * j;
        }
    }

    public static int[][] data() {
        return new int[][] { {1, 2, 2}, {5, 3, 15}, {121, 4, 484} };
    }

    @ParameterizedTest
    @MethodSource(value = "data")
    void testWithMethodSource(int[] data) {
        MyClass myClass = new MyClass();
        int a = data[0];
        int b = data[1];
        int expected = data[2];
        assertEquals(expected, myClass.multiply(a, b));
    }
}

```

In the above test, we declared a method called `data()` which returns the required input values as an `int[][]`. We then passed the name of this method as the value to the `@MethodSource` annotation.

CsvSource

You can create an inline CSV which contains the input for the test and reference that using the `@CsvSource` annotation.

Java

```

import static org.junit.jupiter.api.Assertions.*;

```

```

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

public class UsingParameterizedTest {

    // Class to be tested
    class MyClass {
        // Method to be test
        public int multiply(int i, int j) {
            return i * j;
        }
    }

    @ParameterizedTest
    @CsvSource({"1,2,2", "5,3,15", "121,4,484"})
    void testCsvSource(int a, int b, int expected) {
        MyClass myClass = new MyClass();
        assertEquals(expected, myClass.multiply(a, b));
    }
}

```

CsvFileSource

You can also reference a CSV file by using the `@CsvFileSource` annotation. For example, `@CsvFileSource(resources = "data.csv")`.

OtherSources

ValueSource

You can provide input to the Parameterized test using the `@ValueSource` annotation where you can provide a set of `string`, `long`, `double`, `float` to your test. For example, `@ValueSource(ints = { 1, 2, 3})`.

EnumSource

Lets you pass `Enum` constants as test class. With the optional attribute names you can choose which constants should be used. Otherwise all attributes are used. For example, `@EnumSource(value = Months.class, names = {"JANUARY", "FEBRUARY"})`.

ArgumentsSource

Specifies a class that provides the test data. The referenced class has to implement the `ArgumentsProvider` interface. For example, `@ArgumentsSource(MyArgumentsProvider.class)`.

Exercises

Warm-up exercise

1. Get the warm-up Java project from [here](#).
2. Extract the downloaded file and open the directory on IntelliJ. At this point, you will see a Maven-based Java project with only one class, `pt.up.fe.demo.MyClass`.
3. Create a new test class `MyClassTest`. 🧑 Tip: (Re-)Visit the section [Where should the test be located?](#) in case you do not know where test classes should be placed.
4. Add the following two test cases to your new test class:

Java

```
import static org.junit.jupiter.api.Assertions.assertAll;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class MyClassTest {

    @Test
    public void testExceptionIsThrown() {
        MyClass tester = new MyClass();
        assertThrows(IllegalArgumentException.class,
            () -> tester.multiply(1000, 5));
    }

    @Test
    public void testMultiply() {
        MyClass tester = new MyClass();
        assertEquals(50, tester.multiply(10, 5), "10 x 5 must be 50");
    }
}
```

5. Run your test class on IntelliJ or on the command line with `mvn test`.

6. You might have noticed (either due to a red bar on IntelliJ or due to the `There are test failures` message on the command line) that one of the test cases has failed. Fix the bug in the code under test and re-run the test class to get a green bar / passing test suite.
7. Simplify your test code with `@BeforeEach`. You might have noticed that the initialization of `MyClass` happens in every test. Move the initialization to a `@BeforeEach` method.
8. Define a group assertion with `assertAll`. Define a new test case named `testGrouped`, which checks the conditions in `testExceptionIsThrown` and `testMultiply` at the same time with `assertAll` statement. Change the condition to make both tests fail, run the test and ensure that all checks in the `assertAll` are executed.
9. Adjust the `testGrouped` so that both checks are successfully executed.

Develop unit tests

1. Get the full-exercise Java project from [here](#).
2. Extract the downloaded file and open the directory in IntelliJ. At this point, you will see a Maven-based Java project with a few classes.

Part I

3. Implement the test cases in the test class `pt.up.fe.util.EmailValidatorTest` and ensure that all tests can be successfully executed on IntelliJ or on the command line with `mvn test`.

 Tip: Unfortunately (but on purpose), the test specification is not very good. In any case, the names of the test cases should give you a pointer to what each test aims to test.

4. Choose one of the test cases in the test class `pt.up.fe.util.EmailValidatorTest` and make it run only on the operating system you have installed using `Assumptions`.

 Tip: (Re-)Visit section [How to disable tests / Conditional executions](#) in case you do not know how to use `Assumptions`.

5. Choose one of the test cases in the test class `pt.up.fe.util.EmailValidatorTest` and make it run only on the operating system you have installed using the `@EnabledOnOs` annotation.

 Tip: (Re-)Visit section [How to disable tests / Conditional executions](#) in case you do not know how to use `@EnabledOnOs`.

Part II

6. Develop one parameterized test case in the `pt.up.fe.util.ConverterUtilTest` for the method `convertFahrenheitToCelsius` in the `pt.up.fe.util.ConverterUtil` class with at least three different inputs. You should use the `@MethodSource` annotation.

7. Develop one parameterized test case in the `pt.up.fe.util.ConverterUtilTest` for the method `convertCelsiusToFahrenheit` in the `pt.up.fe.util.ConverterUtil` class with at least three different inputs. You should use the `@CsvSource` annotation.

Part III

8. Implement the test cases in the test class `pt.up.fe.service.DataServiceTest` and ensure that all tests can be successfully executed on IntelliJ or on the command line with `mvn test`.

👤 Tip: The names of the test cases should give you a pointer what each test aims to test.

👤 Tip: There is a bug in the `DataService` class, thus some of the tests should fail. Fix it and then re-run the test suite.

Miscellaneous

- [Java 5 official documentation](#)
- [Growing Object-Oriented Software, Guided by Tests](#)